Paket Desain Arsitektur

Aplikasi Biro Umum dan Rumah Tangga · Kementerian Ketenagakerjaan

Pemilik: Michael · 5 November 2026

Bagian	Isi	Tahap
1	Ringkasan kebutuhan	Tahap 1
2	Catatan keputusan arsitektur	Tahap 2
3	Diagram konteks dan diagram container	Tahap 3
4	Data (ERD) dan integrasi	Tahap 3
5	Akses dan risiko	Tahap 4

- Kode **F** = fitur, **N** = target kualitas, **K** = keputusan, **R** = risiko; dipakai untuk saling merujuk.
- Isi disusun dari kode dan skema database aplikasi yang berjalan. Bagian yang berupa asumsi atau belum diuji ditandai.
- Berkas pendamping: **Michael_Arsitektur.drawio** (bisa diedit; 3 halaman: Konteks, Container, ERD).

Bagian 1 · Ringkasan kebutuhan

Tahap 1 · F = fitur (functional), N = target kualitas (non-functional).

Fitur utama

Kode	Jenis	Kebutuhan
F1	Functional	Pegawai mengajukan permintaan **Pemeliharaan** dan **Pengadaan**; PIC, Kabag, dan Admin memprosesnya lewat tiga tahap berstatus (pending → on_progress → selesai) beserta dokumen.
F2	Functional	Kabag dan Admin mengelola **Kendaraan Dinas** (data, service, dokumen); dashboard menandai pajak yang jatuh tempo dalam 14 hari.
F3	Functional	Tamu masuk dengan akun Google di kiosk lalu mengajukan, mengedit, dan membatalkan **booking ruang rapat** miliknya; sistem menolak jadwal yang bentrok.
F4	Functional	Siapa pun melihat jadwal ruang rapat tanpa login; pegawai mengelola semua booking.

Target kualitas (bisa diuji)

Kode	Jenis	Target dan cara uji
N1	Performance	95% request daftar dan jadwal selesai ≤ 2 detik dengan 1.000 pengguna aktif bersamaan di lingkungan uji (database terpisah berisi data skala produksi). **Target ini asumsi** dan belum diukur; uji beban 4 Nov 2026 pada kode lama menunjukkan lag di sekitar 200 pengguna aktif.
N2	Security	Akun tamu A ditolak (404) saat mengedit atau membatalkan booking tamu B, dan booking B tidak berubah. Uji: dua token tamu.
N3	Security	Karyawan ditolak (403) saat mengedit atau menghapus data Pemeliharaan/Pengadaan; PIC hanya bisa data yang ia buat sendiri. Uji: token karyawan, PIC A, PIC B.

Pengguna

Pengguna	Cara masuk	Peran di sistem
Tamu	Login Google di kiosk	Mengajukan, mengedit, dan membatalkan booking ruang rapat miliknya.
Pegawai (Karyawan, PIC)	Akun aplikasi	Mengajukan Pemeliharaan/Pengadaan, mengelola booking; PIC memproses tahapan data buatannya.
Kabag / Admin	Akun aplikasi	Seperti pegawai, ditambah kendaraan, akun, Mode Maintenance, dan riwayat aktivitas.

Batas scope dan asumsi

- **Di dalam:** empat layanan (Pemeliharaan, Pengadaan, Kendaraan Dinas, Ruang Rapat) beserta login tamu Google dan akun. **Di luar:** integrasi ke sistem pengadaan eksternal, pembayaran sungguhan, dan aplikasi mobile.
- Satu instansi, satu server, satu proses backend. Tamu memiliki akun Google. Target N1 adalah asumsi latihan.
- Email (konfirmasi, pembatalan, pengingat booking) baru ditulis 5 Nov 2026 dan **belum diuji dengan SMTP sungguhan**.

Bagian 2 · Catatan keputusan arsitektur

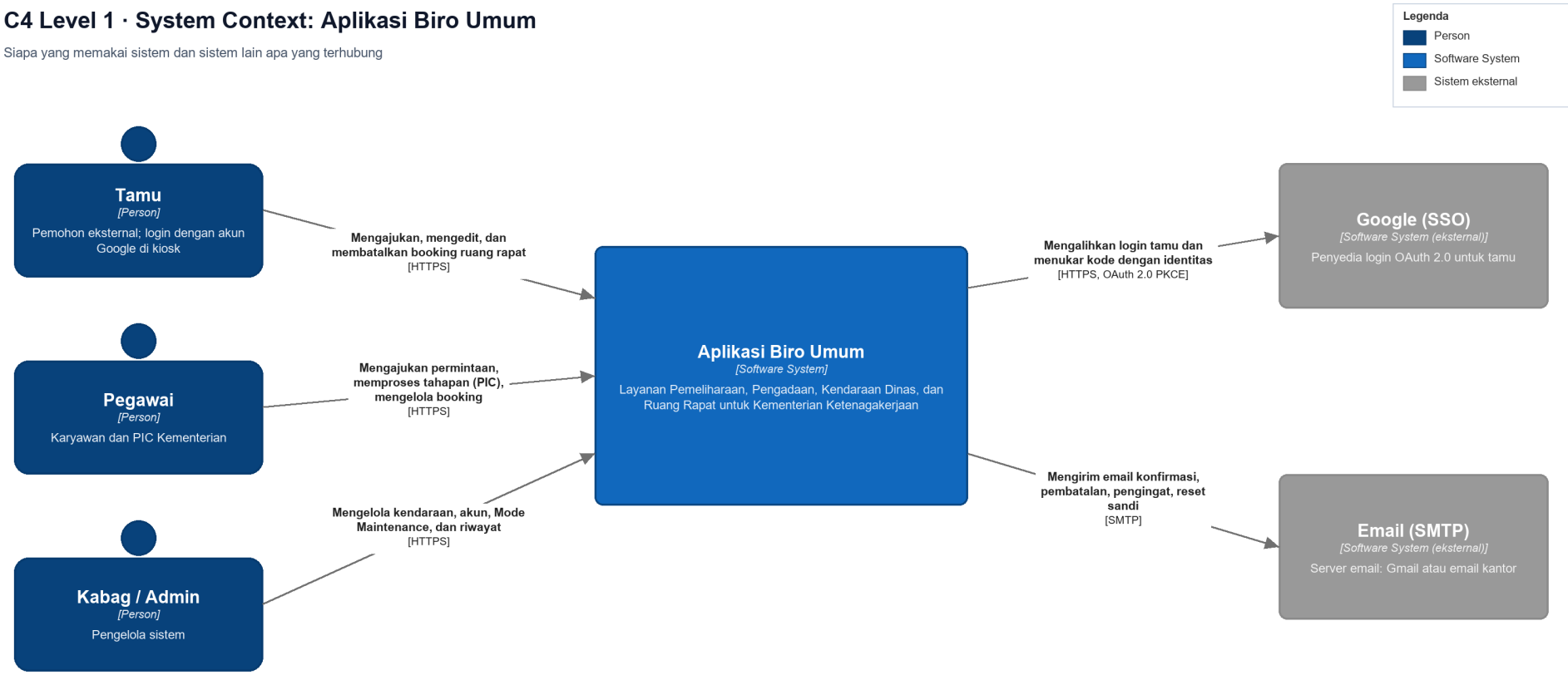
Tahap 2 · pilihan, alasan, konsekuensi, asumsi, dan bagian diagram yang terdampak.

K1 · Modular monolith: satu proses Node.js (Express) dengan modul per fitur, bukan microservices	
Alasan	Satu instansi, tim kecil, satu database. Pembayaran dan booking butuh transaksi dan aturan yang konsisten; satu aplikasi lebih mudah dijalankan dan dipantau.
Konsekuensi	Batas antarmodul belum dijaga ketat: modul tamu memakai fungsi bantu dari modul Ruang Rapat. Backend wajib **satu proses** karena bus live, cache, dan batas laju disimpan di memori; menambah instance berarti memindahkannya ke penyimpanan bersama (mis. Redis).
Asumsi	Beban muat di satu server (lihat N1, belum diukur).
Terdampak	Kotak **API Backend** pada diagram container. Terkait: N1.

K2 · Cegah booking ganda di server dan database, bukan di tampilan	
Alasan	Tanpa kunci, dua request bersamaan sama-sama lolos cek bentrok, dan jadwal di layar bisa basi. Backend memakai transaksi dengan kunci per ruangan (`pg_advisory_xact_lock`) lalu mencari booking yang beririsan.
Konsekuensi	Request untuk ruangan yang sama diproses bergantian. **Tidak ada constraint database yang menjaga bentrok**, jadi aturan ini hanya berlaku bila semua penulisan lewat backend.
Asumsi	Semua perubahan booking dilakukan lewat backend.
Terdampak	Panah API Backend → Database; entity **RUANG_RAPAT**. Terkait: F3.

Bagian 3 · Diagram konteks

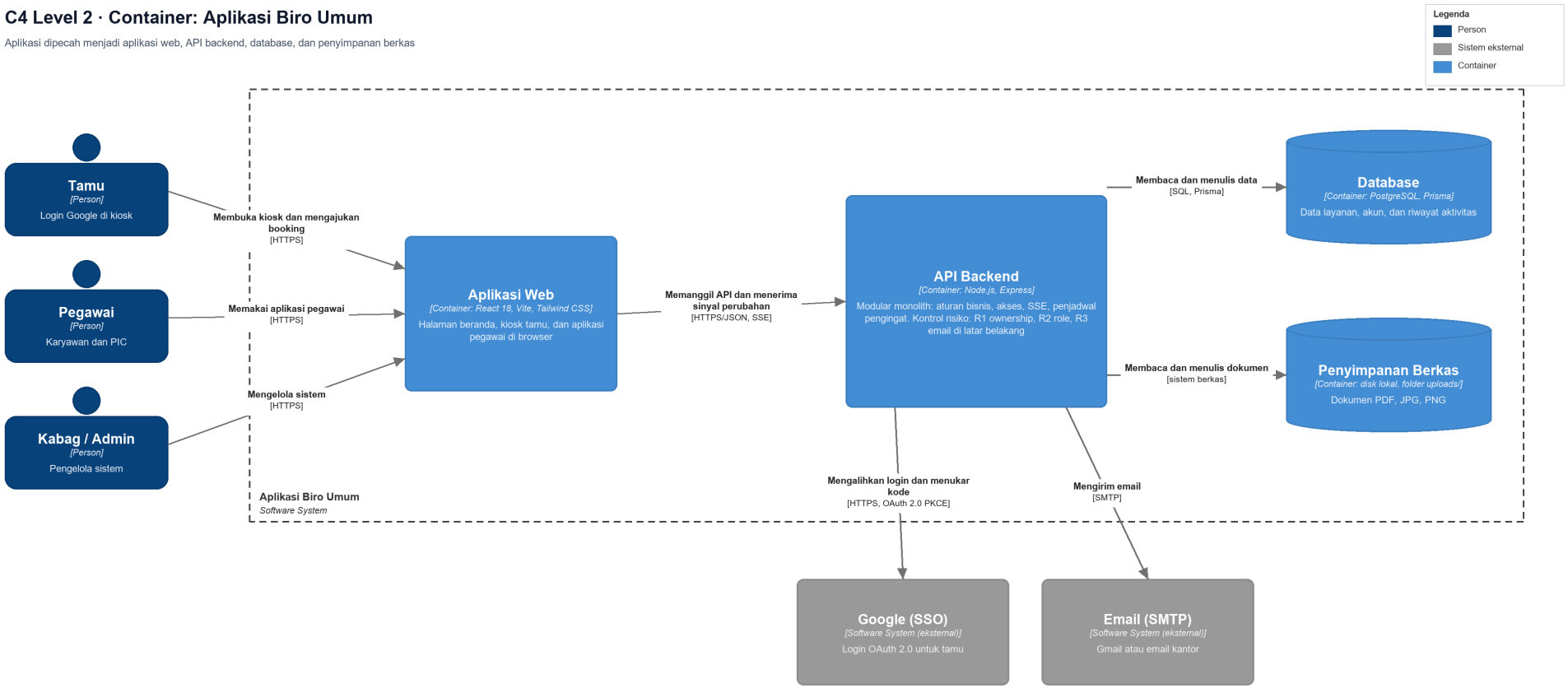
Tahap 3 · aplikasi sebagai satu kotak: pengguna di kiri, sistem lain di kanan.



- Tiga jenis pengguna memakai aplikasi lewat browser. Google mengembalikan identitas tamu setelah login; aplikasi mengirim email lewat SMTP.

Bagian 3 · Diagram container

Tahap 3 · aplikasi dipecah menjadi aplikasi web, API backend, database, dan penyimpanan berkas; label = teknologi.



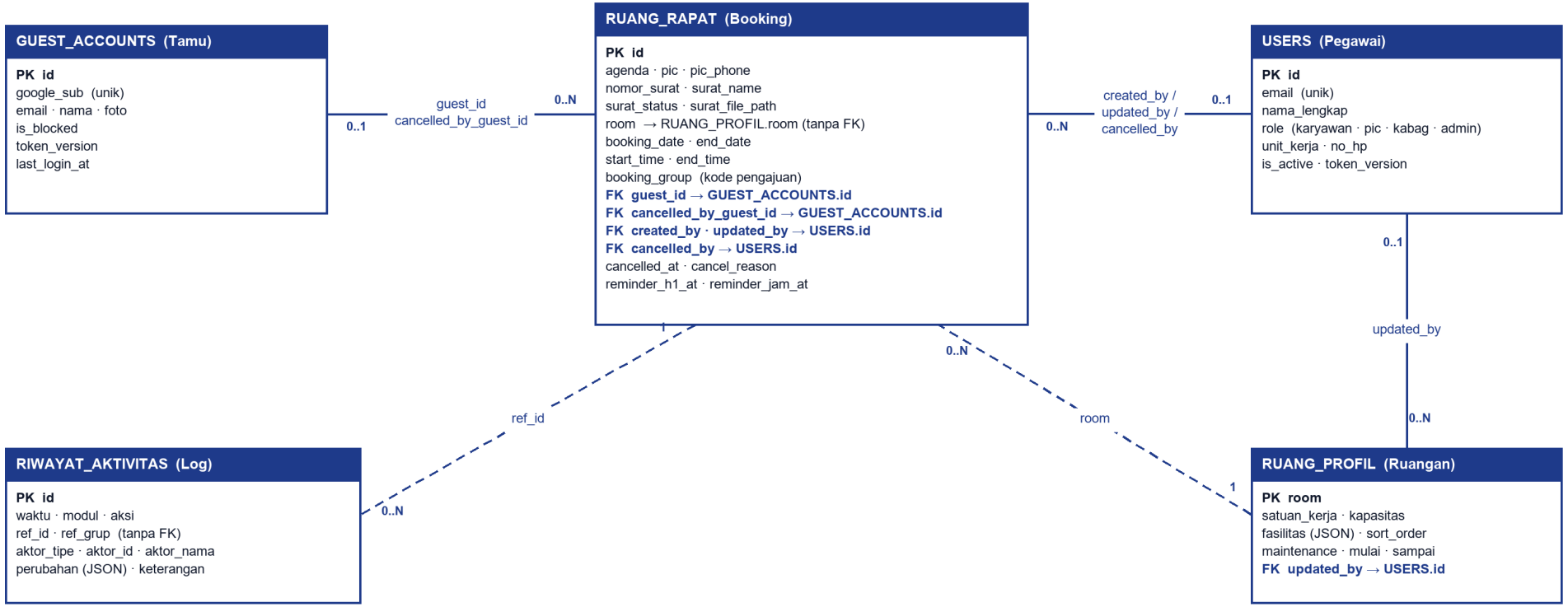
Kontrol risiko berjalan di kotak **API Backend**: R1 cek ownership, R2 cek role dan pemilik data, R3 email di latar belakang (rincian di bagian 5).

Bagian 4 · Data (ERD): Ruang Rapat

Tahap 3 · lima entity; garis putus-putus = rujukan tanpa foreign key (dijaga aplikasi).

ERD 2: Data Booking Ruang Rapat

PK = kunci utama, FK = foreign key. Garis putus-putus = rujukan tanpa FK (dijaga aplikasi)



Satu pengajuan = satu booking_group = satu atau lebih baris RUANG_RAPAT (rentang tanggal berurutan; tanggal loncat-loncat = beberapa baris).
Booking tidak dihapus saat batal: cancelled_at + alasan diisi. Semua FK boleh kosong dan memakai ON DELETE SET NULL.

Bagian 4 · Aturan data dan integrasi

Tahap 3 · setiap relasi dan panah bisa dijelaskan.

Aturan data

- **Satu pengajuan = satu** `booking_group` **= satu atau lebih baris** RUANG_RAPAT (rentang tanggal berurutan; tanggal loncat-loncat menjadi beberapa baris).
- **Bentrok** = booking aktif (`cancelled_at` kosong) di ruangan yang sama dengan tanggal **dan** jam yang beririsan. Dijaga backend (K2), bukan constraint database.
- **Batal tidak menghapus baris:** `cancelled_at`, alasan, dan pembatal (pegawai atau tamu) diisi.
- **Semua FK boleh kosong dengan ON DELETE SET NULL.** `room` bukan FK ke RUANG_PROFIL: kode ruangan dicek aplikasi saat menyimpan.
- **RIWAYAT_AKTIVITAS tanpa FK:** menyimpan sendiri nama, email, dan role aktor serta rujukan data; hanya Kabag/Admin yang bisa membacanya.

Integrasi

Integrasi	Dari → ke	Data	Kapan	Jika gagal
Google OAuth 2.0 (PKCE)	Browser → Google; Backend → Google	Kode otorisasi; token ID berisi email, nama, `sub`.	Tamu klik “Login Google”.	Tamu dialihkan balik dengan pesan galat dan bisa mencoba lagi.
Email (SMTP)	Backend → server SMTP	Penerima, jenis email, dan rincian booking (agenda, ruangan, tanggal, jam).	Setelah ajukan, edit, batal; pengingat berkala.	Dikirim di latar belakang: booking tetap tersimpan, kegagalan dicatat di log, pengingat dicoba ulang maks. 3 kali.
PostgreSQL	Backend → Database (Prisma)	Seluruh data aplikasi.	Setiap request.	Respons 500 dengan pesan umum; detail di log. Penulisan penting memakai transaksi.

Bagian 5 · Akses dan risiko

Tahap 4 · siapa boleh melakukan apa. Baris = peran, kolom = tindakan.

Peran \ Tindakan	Lihat jadwal ruang rapat	Buat booking ruang rapat	Edit / batal booking ruang rapat	Ajukan Pemeliharaan / Pengadaan	Edit / hapus Pemeliharaan / Pengadaan	Lihat data kendaraan	Tambah / ubah / hapus kendaraan	Kelola akun, Maintenance, Riwayat
Tamu (login Google)	Boleh	Boleh	Hanya miliknya	Tidak	Tidak	Tidak	Tidak	Tidak
Karyawan	Boleh	Boleh	Boleh	Boleh	Tidak	Boleh	Tidak	Tidak
PIC	Boleh	Boleh	Boleh	Boleh	Hanya miliknya	Boleh	Tidak	Tidak
Kabag	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh
Admin	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh	Boleh

- **Tamu** masuk lewat login Google di kiosk dan tidak bisa membuka aplikasi internal. “Hanya miliknya” = pengajuan atas akun Googlenya sendiri, selama belum berjalan.
- **Ruang rapat:** semua pegawai sengaja boleh mengelola semua booking, termasuk surat dan statusnya.
- **PIC** adalah karyawan yang otomatis naik peran setelah mengajukan Pemeliharaan/Pengadaan pertamanya. “Hanya miliknya” = data yang ia tambahkan sendiri.
- **Kolom terakhir** mencakup Akun & Akses, Mode Maintenance, Riwayat Aktivitas, serta foto dan daftar ruangan. Kabag setara Admin, kecuali akun **Admin** tidak bisa diubah Kabag.
- Pembatasan diterapkan di **server** (kotak API Backend), bukan hanya dengan menyembunyikan tombol di tampilan.

Bagian 5 · Catatan risiko

Tahap 4 · dua risiko keamanan dan satu risiko sistem gagal; tiap risiko memuat kejadian, dampak, tindakan, cara uji, dan kotak tempat kontrol berjalan.

R1 · Keamanan: tamu mengakses booking milik tamu lain (IDOR)	
Kejadian	ID booking berupa angka berurutan. Tamu yang sudah login mengganti ID pada `PUT /api/guest/bookings/:id` atau `POST /api/guest/bookings/:id/cancel` dengan ID milik tamu lain lalu mengubah atau membatalkannya.
Dampak	Booking orang lain berubah atau batal tanpa izin; ruangan bisa dikosongkan atau diambil; tamu yang dirugikan tidak tahu penyebabnya.
Tindakan	Cek ownership: setiap query selalu menyertakan `guest_id` tamu yang login (bukan hanya ID booking). Booking orang lain dijawab **404**, bukan 403, agar ID yang valid tidak bisa ditebak. Dilakukan di server.
Cara uji	Login sebagai tamu A dan B; buat booking dengan B dan catat ID-nya. Dengan token A panggil PUT edit dan POST cancel ke ID itu. **Benar bila** keduanya 404 dan booking B tidak berubah.
Kotak kontrol	**API Backend** (filter `guest_id`; penanda R1 di diagram container). Kebutuhan: F3, N2.

R2 · Keamanan: pegawai memanggil API di luar wewenang perannya (akses lintas peran)	
Kejadian	Karyawan atau PIC tidak melihat tombol Edit/Hapus, tetapi memanggil API langsung (DevTools atau Postman), mis. `PUT /api/pemeliharaan/:id` milik PIC lain atau `POST /api/kendaraan`.
Dampak	Data pemeliharaan, pengadaan, atau kendaraan orang lain bisa diubah atau dihapus. Riwayat mencatat pelakunya, tetapi kerusakan data sudah terjadi.
Tindakan	Batasi role: `requireRole` di server (Pemeliharaan/Pengadaan: kabag, pic, admin; Kendaraan dan Akun & Akses: admin, kabag) ditambah cek pemilik: PIC hanya boleh bila `created_by` sama dengan ID penggunanya. Peran dibaca dari database tiap request.
Cara uji	Token **karyawan**: PUT dan DELETE ke data Pemeliharaan, lalu POST ke Kendaraan. Token **PIC B**: PUT ke data buatan PIC A. **Benar bila** semuanya 403 dan data tidak berubah; ulangi dengan token Kabag untuk memastikan yang berhak tetap lolos.
Kotak kontrol	**API Backend** (middleware role dan cek pemilik; penanda R2 di diagram container). Kebutuhan: F1, N3.

R3 · Sistem gagal: SMTP bermasalah, email konfirmasi dan pengingat tidak terkirim	
Kejadian	Server email menolak login, kena batas kirim, atau tidak bisa dihubungi. Email konfirmasi, pembatalan oleh Biro Umum, dan pengingat tidak sampai ke tamu.
Dampak	Tamu tidak tahu bookingnya dibatalkan dan datang sia-sia, atau lupa jadwal sehingga ruangan terpakai kosong. Bila pengiriman email ikut menggagalkan simpan booking, tamu tidak bisa memesan sama sekali.
Tindakan	Kirim di latar belakang: kegagalan hanya dicatat di log dan tidak menggagalkan simpan atau batal. Pengingat yang gagal dilepas klaimnya dan dicoba lagi sampai 3 kali. Sumber kebenaran tetap kartu “Booking Anda” di kiosk. Keterbatasan: hitungan percobaan hilang bila backend restart.
Cara uji	Isi `SMTP_PASS` dengan nilai salah, restart backend, lalu ajukan, edit, dan batalkan booking. **Benar bila** semuanya tetap berhasil dan log memuat “Email booking tamu gagal dikirim”. Kosongkan `SMTP_HOST` untuk memastikan fitur email diam tanpa error. **Belum dijalankan:** kode email baru ditulis dan belum diuji.
Kotak kontrol	**API Backend** (pengiriman latar belakang; penanda R3) dan panah ke **Email (SMTP)**. Kebutuhan: F3 (email booking).